



PDF Download
3702138.3702139.pdf
06 April 2026
Total Citations: 0
Total Downloads: 1264

 Latest updates: <https://dl.acm.org/doi/10.1145/3702138.3702139>

RESEARCH-ARTICLE

Generating a Robot Framework Test Script for Web Application Based on Database Constraints

RATTASAN AEKAUKKHARACHAWAKIT, Chulalongkorn University, Bangkok, Thailand

TARATIP SUWANNASART, Chulalongkorn University, Bangkok, Thailand

Open Access Support provided by:

Chulalongkorn University

Published: 29 December 2024

Citation in BibTeX format

ASSE 2024: 2024 5th Asia Service Sciences
and Software Engineering Conference
September 11 - 13, 2024
Tokyo, Japan

Generating a Robot Framework Test Script for Web Application Based on Database Constraints

Rattasan Aekaukkharachawakit
Department of Computer Engineering Faculty of
Engineering
Chulalongkorn University
Bangkok, Thailand
rattasando@gmail.com

Taratip Suwannasart
Department of Computer Engineering Faculty of
Engineering
Chulalongkorn University
Bangkok, Thailand
taratip.s@chula.ac.th

Abstract

In software development, software testing is an essential process to identify defects in software and enhance product reliability. Manual testing can be useful for spotting defects; however, it requires software testers to thoroughly understand the software's functionality, which can be resource-intensive, particularly in terms of time. With the availability of automation testing tools today, testing can be expedited. Nonetheless, software testers must comprehend the complex software frameworks developers use and the database system's constraints. This often leads to a significant time investment. Generating a test script, which requires understanding the automation framework and the format of the combined test data, is a time-consuming process. The proposed approach makes it easier for testers to create test scripts and test data, ensuring that the test data satisfies both front-end and database constraints. It consists of a front-end JavaScript component to analyze the inputs that can be entered into web pages to create test scripts, and a back-end Spring Framework project to analyze the entities and connect to the tested database to implement database constraints for analysis to create test data. By utilizing the provided inputs, the research can assess boundary values and database constraints to produce appropriate test data and scripts.

CCS Concepts

• **Software and its Engineering**; • **Software creation and management**; • **Software verification and validation**; • **Software defect analysis**; • **Software testing and debugging**;

Keywords

Software Testing, Test Automation, Data-Driven Testing, Robot Framework, Test Script, Database Constraints

ACM Reference Format:

Rattasan Aekaukkharachawakit and Taratip Suwannasart. 2024. Generating a Robot Framework Test Script for Web Application Based on Database Constraints. In *2024 5th Asia Service Sciences and Software Engineering*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASSE 2024, September 11–13, 2024, Tokyo, Japan

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1754-3/24/09

<https://doi.org/10.1145/3702138.3702139>

Conference (ASSE 2024), September 11–13, 2024, Tokyo, Japan. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3702138.3702139>

1 Introduction

The software industry is experiencing significant growth, making web applications increasingly popular for business purposes. Databases play a crucial role in storing data for software, enabling it to function effectively and reliably. Ensuring software quality requires a software testing process, which can be resource-intensive in terms of both labor and time. Testing requires knowledgeable and skilled testers who must create test cases that encompass all functionalities of the software and aim to identify errors within the system. These tasks often involve manually creating test cases, making software testing a repetitive and time-consuming process. Automation testing [1] is one approach to reduce resource usage in software testing. By replacing repetitive manual testing with automated processes, software testers can conduct tests more efficiently according to predefined steps and activities. However, creating test scripts for automated testing still requires experience and an understanding of programming languages, making the process challenging and complex for testers.

Today, automated software testing is facilitated by Test Automation Frameworks [2] such as Robot Framework [4], Cypress, Cucumber, and Citrus. These frameworks help software testers by providing various types of testing methods, such as Keyword-Driven Testing, Hybrid Testing, and Data-Driven Testing [3]. Due to their reusability and simplicity, this research opts for test scripts and test data created using a data-driven testing framework. This approach allows testers to edit test steps within the test script more easily, as it uses a single test script and imports test data from external files. To create test scripts, testers commonly define the Element Locator, used with the Selenium Library [7] to create test case input sequences. Locator element values are derived from analyzing HTML [5] attributes such as id, name, class, and others. Testers can also specify conditions for inputting data, such as maximum and minimum values. However, web application development may not always specify input conditions explicitly. Analyzing only HTML data and input conditions from web pages may not provide sufficient test data that aligns with database boundary values or constraints. This can lead to issues when using the created test script, resulting in errors and undermining the reliability of the software testing process. To address these challenges, this research proposes an approach to create test scripts that relate to the frontend, backend, and database constraints of web applications, aiming to improve the accuracy and effectiveness of automated testing.

Databases play a crucial role in storing data received from an application, especially when dealing with large and complex systems. For instance, a web application for managing employee information might involve forms where details such as first name, last name, employee identification number, and email address are entered. Some inputs may have conditions that prevent duplicate data from being saved if it already exists in the database. Therefore, when creating test data, it is essential to check the data in the database to ensure that it adheres to the database's constraints and existing records. To effectively test web applications, software testers need to be skilled in creating test scripts that consider both the frontend and backend components of the application, as well as the database constraints. This comprehensive approach ensures that the test scripts can reliably evaluate the application and uncover any potential issues.

Related research by Panthakan Sriwichainan and Taratip Suwannasart [12] introduced a method for creating tests for web applications by analyzing HTML data and generating test scripts that initially lack test data. They used XSD data to identify and extract edge values, which were then used to create test data. The generated test data was subsequently added to the test script. Although the test data produced from XSD analysis may not fully align with the database, the concept of creating test scripts and test data remains valuable and applicable. This method demonstrates the potential of using data analysis techniques to inform the creation of effective test scripts and data for web application testing.

Kartik Bajaj [13] introduced techniques for generating HTML input elements in web applications and discussed using applied mathematical models with open-source tools. His tool, Live Editor for DOM (LED), offers the ability to search for IDs, classes, and locator types within HTML elements. The techniques and tools presented in his research can be leveraged to create HTML input elements for testing during the process of generating test scripts. This approach highlights the potential of combining advanced analysis methods with open-source tools to enhance the efficiency and effectiveness of test script creation.

Additionally, Satish Gojare [14] proposed an automated testing framework for testing web applications using Selenium WebDriver. The framework leverages TestNG to generate summary reports and capture screenshots of test results. The concepts from the aforementioned research on automated testing frameworks for web applications can be integrated into our study. Our research introduces a method for creating test scripts for web applications based on database constraints, with the goal of aiding testers in generating test scripts that incorporate information stored in the database.

In this paper, we propose an approach to generating a Robot Framework test script for a web application based on database constraints. Initially, creating robot test scripts and test data for web applications involves analyzing the HTML input elements displayed on the front-end web page. This analysis is achieved by utilizing a JavaScript file to categorize the structure of the imported data into conditions and boundary values, guiding the creation of the test script. Subsequently, the Spring RESTful API is analyzed to understand the entities and their mapping to the front end. A connection is established with the tested program's database to consider its scope, constraints, and the boundary values from the

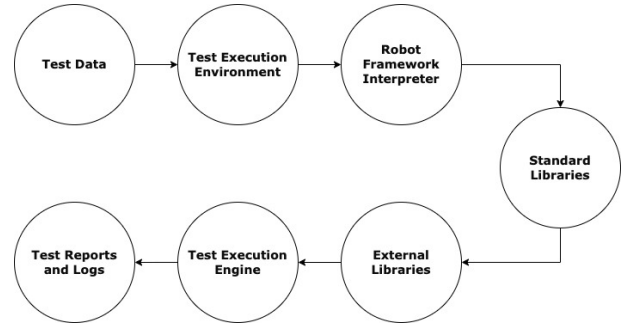


Figure 1: Robot framework architecture.

web application. This process includes extracting data for creating test cases and generating test data. The overall goal is to ensure that the test scripts and test data align with the database constraints and provide the most comprehensive software testing possible.

This paper is organized as follows. Section 2 explains the background in detail, covering data-driven test automation, HTML elements, the robot framework (an automation framework for running test scripts), methods for generating test data with robust boundary value analysis, and database constraint requirements that allow operations to be performed on the database. Section 3 depicts the proposed approach, while Section 4 presents our experiments. Finally, Section 5 summarizes the conclusions and future work.

2 Background

2.1 Data Driven Testing

A data-driven testing [3] approach involves automating tests using keywords, with testing constraints incorporated into the script. Test data is organized separately in a table-format file. This separation between the test script and test data file allows for reusability in future testing, thereby saving a significant amount of time and effort.

2.2 Robot Framework

Robot Framework [4] is a framework for automated testing that uses keyword-driven test scripts. Additionally, it can connect to test libraries from external sources, providing access to more keywords. The architecture of the Robot Framework is shown in Figure 1.

In this section, we will only discuss test data. Test data is a set of data used for testing software. It is designed with specific syntax by a software tester, who must define the following elements:

- 1.Settings: These specify the target test library.
- 2.Variables: These are defined for use in the test scripts.
- 3.Test Cases: These specify the test cases that relate to software requirements.
- 4.Keywords: These are custom definitions created by the software tester for use in test scripts.

2.3 HTML

HTML (HyperText Markup Language) is a language used for creating web pages. It is developed and maintained by the World Wide Web Consortium (W3C) [5]. HTML enables the connection

Table 1: Examples of HTML elements categorized by input fields.

HTML Element group	Input fields
Selection element	Select (drop-down list)
Input element	Date
	Email
	Tel
	Number
	Text
Button Element	Checkbox
	Button
	Radio
	Reset
	Submit

to other web pages and stores data in hypertext format. It uses tags to define the structure of content, controlling how it is displayed in web pages through a web browser.

2.4 HTML Elements

An HTML element is an object that is displayed in a web browser and is defined by the World Wide Web Consortium [6]. The components of an HTML element are presented in Table 1.

In this research, the following HTML elements and input fields will be utilized:

- Selection element: Drop-down list.
- Input elements: Text, telephone, email, number, and checkbox.
- Button elements: Submit button and radio button.

2.5 Selenium

The Selenium library [7] is an external library of the Robot Framework used for testing web applications. The Selenium driver is a module that controls web browsers. Additionally, all keywords in the Selenium library interact with components of a web application. The Selenium library utilizes element locators defined in test scripts. Element locators are shown in Table 2.

In this research, the related element locators used will be ID and Name:

•ID: This is an element that can have a unique value assigned to it. Changing the element locator does not affect the test scripts, such as locator :: id=u_0_n.

•Name: This is an element that can have a name assigned to it, such as locator :: name=lastname.

The Selenium keywords for test scripts are shown in Table 3.

2.6 Boundary Value Testing

Boundary value testing [8] is a test case design technique that uses well-defined boundaries of input data to generate test data, with the goal of detecting potential errors in the input data. This approach includes Robust Boundary Value Testing (RBVT), which focuses on creating test data based on seven input variables.

- “Min-”: The boundary value that is one less than the smallest boundary value.
- “Min”: The smallest boundary value.
- “Min+”: The boundary value that is one greater than the smallest boundary value.
- “Nom”: The normal value within the range.
- “Max-”: The boundary value that is one less than the maximum boundary value.
- “Max”: The greatest boundary value.
- “Max+”: The boundary value that is one greater than the maximum boundary value.

This testing approach operates under the Single Fault Assumption, where a program error is caused by one invalid value while the remaining values are valid. By focusing on these boundary values, the testing aims to identify and rectify potential issues with input data.

2.7 Database Constraints

Allowing operations to modify data stored in tables, such as INSERT, UPDATE, or DELETE statements, must adhere to conditions defined in the database. The Relational Database Management System (RDBMS) verifies whether these operations violate existing conditions based on the following integrity constraints [9] :

•Domain Constraint ensures that stored data has the correct data type and conforms to the permissible values defined in the database.

- Key Constraint ensures that key values used for storage are unique (UNIQUE) and not NULL.
- Entity Integrity Constraint requires that the PRIMARY KEY value is not NULL (NOT NULL).
- Referential Integrity Constraint ensures that the relationship between two tables has a FOREIGN KEY value that references a PRIMARY KEY value from another table.

These constraints are specified using SQL language keywords:

Table 2: Examples of element locator strategies.

Strategies	Match based on	Examples
id	Element <u>id</u>	id:example
name	<u>name</u> attributes	name:example
identifier	Either <u>id</u> or <u>name</u>	identifier:example
class	Element <u>class</u>	class:example
tag	Tag name	tag:div
xpath	XPath expression	xpath://div[@id="example"]

Table 3: The examples of html element types and selenium keywords.

Html element types	Selenium keywords
text, tel, email, number	Input text “locator” “text”
select	Select From List By Value “locator” “value”
radio	Select Radio Button “group_name” “value”
checkbox	Select Checkbox “locator”
submit	Click Element

- **PRIMARY KEY Constraint:** Ensures that data in the column must be unique and cannot be NULL.
- **FOREIGN KEY Constraint:** A column in a table that references a PRIMARY KEY value from another table.
- **UNIQUE Constraint:** Requires that data in the column must be unique.
- **NOT NULL Constraint:** Requires that data in the table must not be NULL. If any of these conditions are violated, the relational database management system will respond with an error.

CHECK Constraint: Allows specifying a range of acceptable data that can be stored in a table, such as “18 < age ≤ 60”. CHECK is available in MySQL, an RDBMS.

In this research, the conditions considered include PRIMARY KEY, UNIQUE, NOT NULL, and CHECK constraints in SQL language.

2.8 Spring Framework

The Spring Framework [10] is a framework that helps software developers create applications in Java for various platforms such as web applications, enterprise applications, standalone applications, and web services. These types of applications can be complex and challenging to set up for development. To streamline the development process, the Spring Boot framework, which builds upon the Spring Framework, provides a simpler way to set up the environment for building applications.

2.9 Spring Data REST

Spring Data REST [11] is one of the projects under the Spring Data umbrella that focuses on creating RESTful web services to support REST APIs. Therefore, this research will focus on Spring Data REST.

3 Approach

This research introduces a method for analyzing two key components: front-end JavaScript files and back-end paths built with the Spring framework. This section outlines the analysis details. The method aims to create a tool that generates Robot Framework test scripts and corresponding test data aligned with both front-end conditions and database constraints. Figure 2 illustrates the overall research approach, which involves these key steps:

3.1 Input Receiver

This module accepts input from the software tester, including the paths to the frontend file and backend project.

3.2 Analyze HTML Input Elements

In the web application, HTML elements must be displayed to allow users to view and input data. These elements handle data input based on the type of element that must be filled, such as numbers, characters, letters, or selections. The JavaScript file of the web application, developed using the React library, contains a render function that stores the HTML source code for HTML elements, as shown in Figure 3. This research will analyze front-end JavaScript files to extract HTML elements and input conditions from the <input/> tag. HTML element data can be extracted, including attributes such as id, type, and required. For numeric data types, the max and min values are analyzed. For text data, the maxLength and minLength attributes are considered. For select item types, the value attribute is analyzed. The tool stores the results of extracting HTML elements and conditions, as shown in Table 4.

3.3 Generate Robot Test Script

A Robot Framework test script can be created by taking the input IDs from Table 4 and using them to define the locator element, which represents the location of each HTML element displayed on the web application’s screen. The input type is used as Selenium keywords as shown in Table 5. The results of creating the Robot Framework test script are shown in Figure 4.

3.4 Analyze Http Method

In this step, we analyze HTTP request methods in a REST API architecture. The front end uses the Axios library to send requests to the back end. This information can be analyzed from the front-end JavaScript file, as illustrated in Figure 5.

3.5 Extract Database Connection

In this step, the backend project developed with the Spring Framework contains a file called application.properties that sets up the database connection. This file can be used to extract database connection information, including the URL, username, password, and database name, as shown in Figure 6.

3.6 Extract Database Constraints

In this step, the database constraints will be extracted using SQL from the table information_schema.table_constraints, which contains the database’s constraints data. Then, the SQL command SHOW CREATE TABLE ‘employees’ is used to show details of the table such as field names, data types, constraints, and more, as shown in Figure 7.

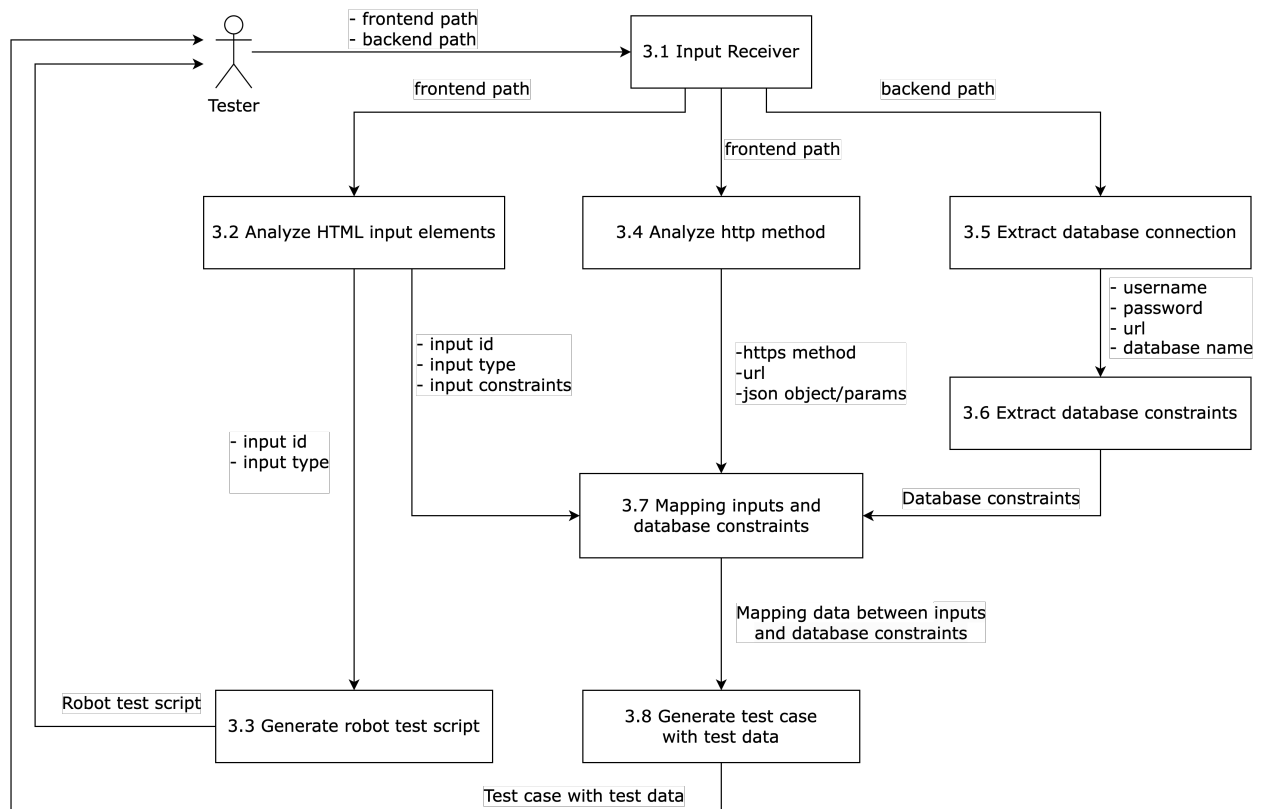


Figure 2: Structure of the tool.

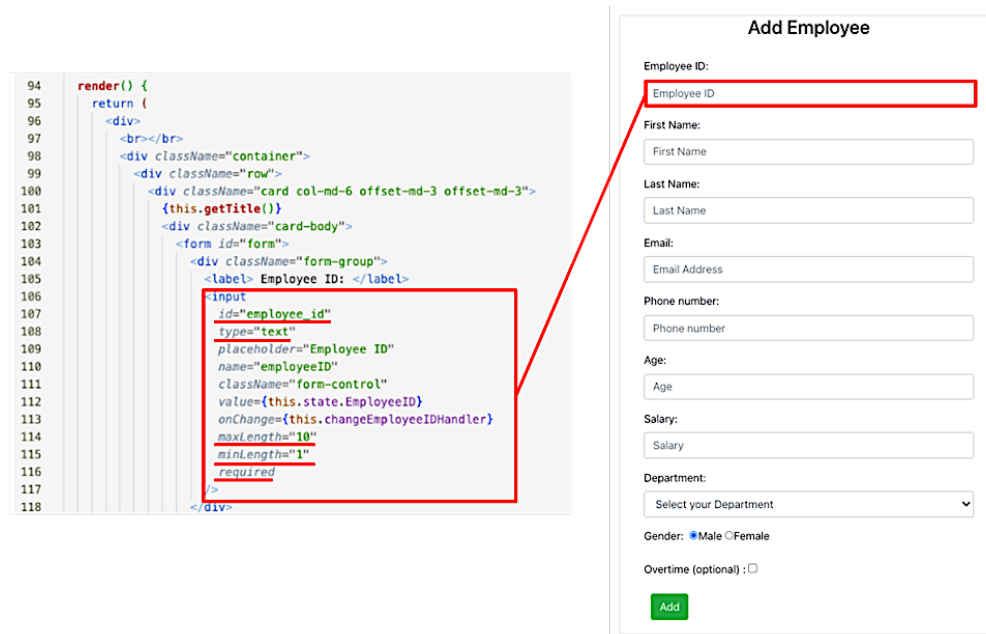


Figure 3: JavaScript and User interface in browser.

Table 4: The details of the HTML input element are as follows.

HTML id	HTML type	minLength, maxLength	min, max	value	required
employee_id	text	1, 10	-	-	Y
first_name	text	1, 50	-	-	Y
last_name	text	1, 50	-	-	Y
email	email	1, 50	-	-	Y
phone_number	tel	1, 10	-	-	Y
age	number	-	18, 60	-	Y
salary	number	-	15000, 100000	-	Y
department	select	-	-	Accountant, Developer	Y
gender	radio	-	-	Male, Female	Y
overtime	checkbox	-	-	T,F	N
add	submit	-	-	-	N

Table 5: HTML and Selenium keyword creation results.

Steps	HTML id	HTML type	Selenium keyword
1	employee_id	text	Input Text
2	first_name	text	Input Text
3	last_name	text	Input Text
4	email	email	Input Text
5	phone_number	tel	Input Text
6	age	number	Input Text
7	salary	number	Input Text
8	department	select	Select From List By Value
9	gender	radio	Select Radio Button
10	overtime	checkbox	Select Checkbox
11	add	submit	Click Element

3.7 Mapping Inputs and Database Constraints

The inputs from steps 3.2, 3.4, and 3.6 are analyzed to establish a relationship between the frontend input and the database columns, creating a mapping of HTML elements, entity classes, and database constraints. This mapping table can be structured as shown in Table 6

3.8 Generate Test Case with Test Data

The process of creating test data is a crucial part of ensuring the accuracy and reliability of web applications. In this research, we focus on creating test data that aligns with database constraints and ensures comprehensive testing coverage. By utilizing various methods to generate test data for different types of HTML elements, we aim to test the web application effectively and identify potential issues. The following steps outline our approach to generating test data for web application testing:

- 1. Creating Test Data: For a group of HTML elements such as selection and buttons, including select, checkbox, and radio elements, test data will be generated using a random value method.

- 2. Creating Test Data: For a group of HTML elements for input such as text, tel, email, and number fields, test data is created using:
- Robust Boundary Value Testing: A method focused on testing the boundaries of input values.
- Database Sampling: Randomly drawing data from the database.
- Database Constraints: Generating test data in accordance with database constraints.

The generated test data adheres to the single fault assumption and is saved in a CSV file.

Figure 8 illustrates test cases contained within the test data file. The test cases are designed to validate the web application using a variety of input scenarios based on the data provided in the test data file.

From Figure 8, the following explanations are provided:

- Test case IDs 1, 2, and 3 contain test data that does not violate database constraints and boundary values.
- Test case ID 4 uses the RBVT (Robust Boundary Value Testing) method for the employee_id input, with an input character length of 0. This represents the only faulty min/min-boundary value in this test case.

```

1  *** Settings ***
2  Library      SeleniumLibrary
3  Library      BuiltIn
4  Library      String
5  Suite Teardown    Close Browser
6  Library      DataDriver      "CSV Path"
7  Test Template    Testing Input Http POST
8  *** Test Cases ***
9  Run | Debug | Run in Interactive Console
10 Testing Input Http POST    ${employee_id} ${first_name} ${last_name} ${email}
11 ... ${phone_number} ${age} ${salary} ${department} ${gender}
12 Load in Interactive Console
13 *** Variables ***
14 ${url}      "URL"
15 *** Keywords ***
16 Load in Interactive Console
17 Testing Input Http POST
18 [Arguments] ${employee_id} ${first_name} ${last_name} ${email}
19 ... ${phone_number} ${age} ${salary} ${department} ${gender}
20 Open Browser      about:blank      chrome
21 Go To             ${url}
22 Input Text        /*[@id="employee_id"] ${employee_id}
23 Input Text        /*[@id="first_name"]  ${first_name}
24 Input Text        /*[@id="last_name"]   ${last_name}
25 Input Text        /*[@id="email"]       ${email}
26 Input Text        /*[@id="phone_number"] ${phone_number}
27 Input Text        /*[@id="age"]         ${age}
28 Input Text        /*[@id="salary"]      ${salary}
29 Select From List By Value /*[@id="department"] ${department}
30 Select Radio Button      gender ${gender}
31 Select Checkbox /*[@id="overtime"]
32 Click Element /*[@id="add"]
33 Close Browser

```

Figure 4: Result of the generated robot test script.

```

73  axios
74  .post("http://localhost:8080/api/v1/employees", {
75    employeeID: this.state.EmployeeID,
76    firstName: this.state.FirstName,
77    lastName: this.state.LastName,
78    emailId: this.state.EmailID,
79    age: this.state.Age,
80    salary: this.state.Salary,
81    department: this.state.Department,
82    phoneNumber: this.state.PhoneNumber,
83    gender: this.state.Gender,
84    overtime: this.state.Overtime,
85  })

```

Figure 5: Http request method in JavaScript file.

- Test case ID 5 uses the RBVT method for the employee_id input, with an input character length of 11. This represents the only faulty max+ boundary value in this test case.
- Test case ID 6 uses the RBVT method for the age input, with an input value of 17. This represents the only faulty min-boundary value in this test case.
- Test case ID 7 uses the RBVT method for the salary input, with an input value of 100,001. This represents the only faulty max+ boundary value in this test case.
- Test case ID 8 uses a random draw method from the database for the employee_id input, resulting in a value of a000000001.

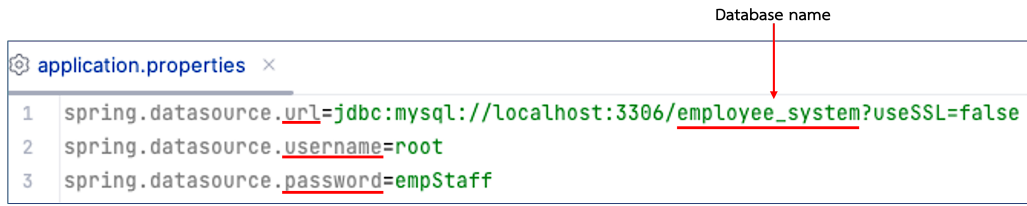


Figure 6: Application properties file.

```

1 CREATE TABLE `employees` (
2   `employee_id` varchar(10) NOT NULL,
3   `first_name` varchar(50) NOT NULL,
4   `last_name` varchar(50) NOT NULL,
5   `phone_number` varchar(10) NOT NULL,
6   `email_id` varchar(50) NOT NULL,
7   `age` int NOT NULL,
8   `salary` int NOT NULL,
9   `department` varchar(50) NOT NULL,
10  `gender` tinyint NOT NULL,
11  `overtime` tinyint NOT NULL,
12  PRIMARY KEY (`employee_id`),
13  UNIQUE KEY `phone_number_UNIQUE` (`phone_number`),
14  UNIQUE KEY `email_id_UNIQUE` (`email_id`),
15  CONSTRAINT `employees_chk_1` CHECK (18 <= `age`),
16  CONSTRAINT `employees_chk_2` CHECK (`age` <= 60),
17  CONSTRAINT `employees_chk_3` CHECK (15000 <= `salary`),
18  CONSTRAINT `employees_chk_4` CHECK (`salary` <= 100000)
19 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

```

Figure 7: Table information and database constraints displayed as Data Definition Language (DDL).

Table 6: Mapping data between data inputs and database constraints.

HTML id	Database column name	HTML type	minLength, maxLength	min, max	value	required	Data type in Database	Database constraints
employee_id	employee_id	text	1, 10	-	-	Y	VAR-CHAR(10)	PK, NN
first_name	first_name	text	1, 50	-	-	Y	VAR-CHAR(50)	NN
last_name	last_name	text	1, 50	-	-	Y	VAR-CHAR(50)	NN
email	email_id	email	1, 50	-	-	Y	VAR-CHAR(50)	UQ, NN
phone_number	phone_number	tel	1, 10	-	-	Y	VAR-CHAR(10)	UQ, NN
age	age	number	-	18, 60	-	Y	INT	CHECK, NN
salary	salary	number	-	15000, 100000	-	Y	INT	CHECK, NN
department	department	select	-	-	Accountant, Developer	Y	VAR-CHAR(50)	NN
gender	gender	radio	-	-	Male, Female	Y	TINYINT	NN
overtime	overtime	checkbox	-	-	T,F	N	TINYINT	NN
add	-	submit	-	-	-	-	-	-

TC ID	\$(employee_id)	\$(first_name)	\$(last_name)	\$(email)	\$(phone_number)	\$(age)	\$(salary)	\$(department)	\$(gender)	\$(overtime)	Expected Output
1	a000000005	Tom	Kim	tom.kim@test.com	0900010005	30	50000	Accountant	Male	T	valid
2	a000000006	Timmy	Arison	timmy.arison@test.com	0900010006	30	50000	Developer	Male	T	valid
3	a000000007	Jackson	Perky	jackson.perky@test.com	0900010007	30	50000	Accountant	Male	T	valid
4		Amy	Jerry	amy.jerry@test.com	0910010008	30	50000	Developer	Female	F	invalid
5	a000000008	Jack	Morgan	jack.morgan@test.com	0910010009	30	50000	Developer	Male	T	invalid
6	a000000009	Vardy	Tim	vardy.tim@test.com	0910010010	17	50000	Accountant	Male	F	invalid
7	a000000010	Lisa	Diya	lisa.diya@test.com	0910010011	30	100001	Accountant	Female	F	invalid
8	a000000001	Sam	Hanna	sam.hanna@test.com	0910010012	30	50000	Developer	Male	T	invalid
9	a000000011	Darren	Fatima		0910010013	30	50000	Developer	Male	F	invalid
10	a000000012	Mango	Rice	mike.shell@test.com	0910010014	30	50000	Developer	Male	F	invalid
11	a000000013	Focus	Charles	focus.charles@test.com	0900010001	30	50000	Accountant	Female	T	invalid

Figure 8: A File containing test cases.

employee_id	first_name	last_name	phone_number	email_id	age	salary	department	gender	overtime
a000000001	Mike	Shell	0900010001	mike.shell@test.com	28	18000	Accountant	0	0
a000000002	Mix	Alpha	0900010002	mix.alpha@test.com	25	15004	Accountant	0	0
a000000003	Alvin	Harry	0900010003	alvin.harry@test.com	22	20000	Developer	1	0
a000000004	Talon	Happy	0900010004	talon.happy@test.com	18	99999	Developer	1	0

Figure 9: Data in the database.

The only fault in this test case is that the value violates primary key database constraints.

- Test case ID 9 generates null values for test data, causing conflicts with database constraints.
- Test case ID 10 uses a random draw method from the database for the email input, resulting in a value of mike.shell@test.com. The only fault in this test case is that the value violates unique key database constraints.
- Test case ID 11 uses a random draw method from the database for the phone_number input, resulting in a value of 0900010001. The only fault in this test case is that the value violates unique key database constraints.

The data highlighted in the box represents data from the database that was randomly drawn to create test data in the previous step. This information is shown in Figure 9.

The tool we developed is shown in Figure 10. It is a Java desktop application that features a section for editing the database connection and specifying additional program output.

4 Experiment

In this section, the results of running the test script obtained from Section 3.3, which added some Selenium keyword "Alert Should Be Present" to verify the test case result, and the test data obtained from Section 3.8 are used. The results of the run are saved as "report.html" as shown in Figure 11, and "log.html" as shown in Figure 12.

The database now includes test data for valid test case IDs 1, 2, and 3, which are highlighted in the box, as depicted in Figure 13.

Figure 14 illustrates the test cases derived from the related work, presenting valid test scripts created in the keyword-driven format. The test data generation process does not take into account constraints, such as unique constraints for input email. It's clear from the test log depicted in Figure 15 that the second valid test case fails.

Figure 10: The user interface of the tool.

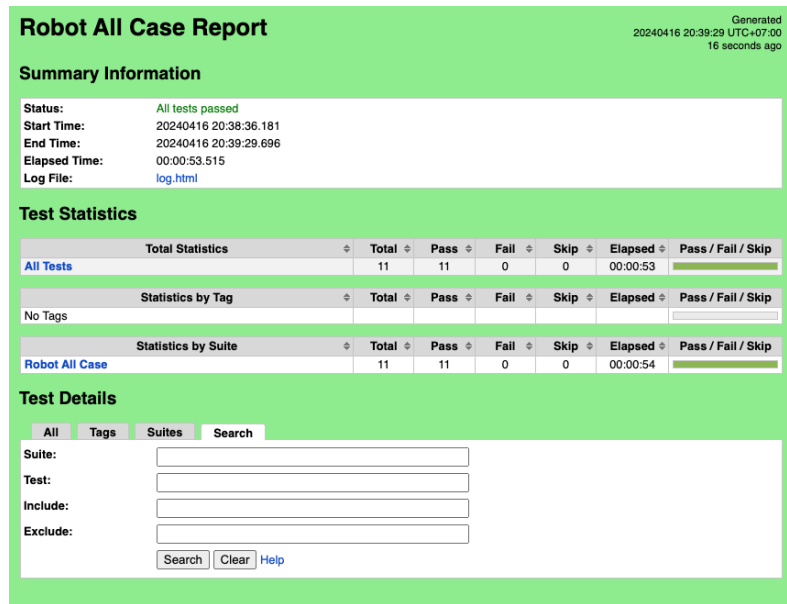


Figure 11: Test report result.

Test Execution Log

SUITE Robot All Case		00:00:55.711
Full Name: Robot All Case		
Source: /Users/dodo/Development/ROBOT/paper/robot_all_case.robot		
Start / End / Elapsed: 20240416 20:06:55.362 / 20240416 20:07:51.073 / 00:00:55.711		
Status: 11 tests total, 11 passed, 0 failed, 0 skipped		
TEARDOWN SeleniumLibrary Close Browser		00:00:00.001
TEST Testing Input Http POST		00:00:05.368
Full Name: Robot All Case.Testing Input Http POST		
Start / End / Elapsed: 20240416 20:06:55.651 / 20240416 20:07:01.019 / 00:00:05.368		
Status: PASS		
KEYWORD Testing Input Http POST a000000005, Tom, Kim, tom.kim@test.com, 0900010005, 30, 50000, Accountant, Male		00:00:05.367
TEST Testing Input Http POST		00:00:04.630
TEST Testing Input Http POST		00:00:05.025
TEST Testing Input Http POST		00:00:05.062
TEST Testing Input Http POST		00:00:05.273
TEST Testing Input Http POST		00:00:05.336
TEST Testing Input Http POST		00:00:05.134
TEST Testing Input Http POST		00:00:04.782
TEST Testing Input Http POST		00:00:04.812
TEST Testing Input Http POST		00:00:04.820
TEST Testing Input Http POST		00:00:05.167

Figure 12: Test log result.

employee_id	first_name	last_name	phone_number	email_id	age	salary	department	gender	overtime
a000000001	Mike	Shell	0900010001	mike.shell@test.com	28	18000	Accountant	0	0
a000000002	Mix	Alpha	0900010002	mix.alpha@test.com	25	15004	Accountant	0	0
a000000003	Alvin	Harry	0900010003	alvin.harry@test.com	22	20000	Developer	1	0
a000000004	Talon	Happy	0900010004	talon.happy@test.com	18	99999	Developer	1	0
a000000005	Tom	Kim	0900010005	tom.kim@test.com	30	50000	Accountant	1	1
a000000006	Timmy	Arison	0900010006	timmy.arison@test.com	30	50000	Developer	1	1
a000000007	Jackson	Perky	0900010007	jackson.perky@test.com	30	50000	Accountant	1	1

Figure 13: Data in the database after running the test script.

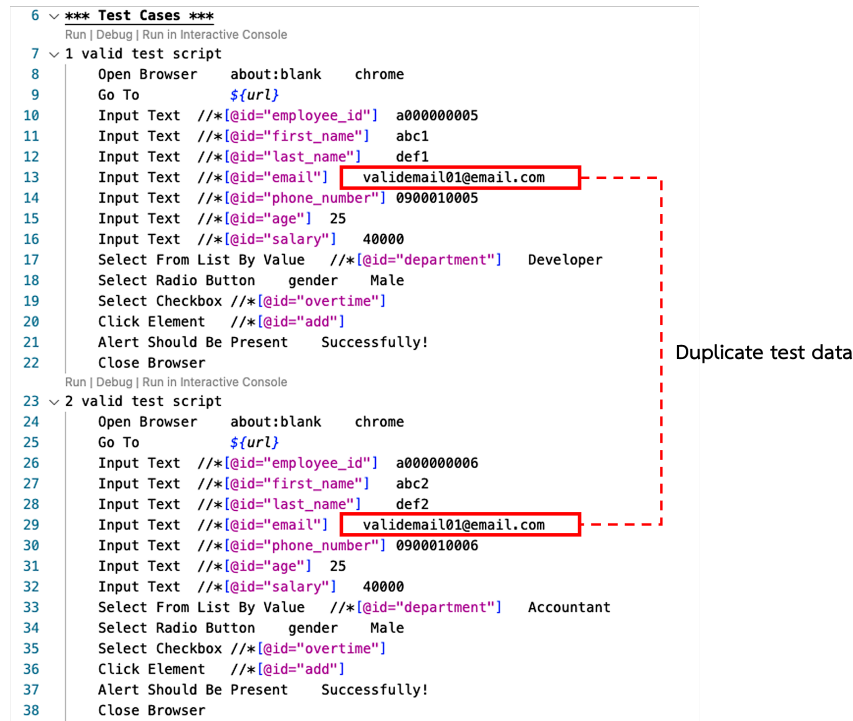


Figure 14: Test script from related work.

Test Execution Log REPORT

SUITE	Robot Related Work	00:00:24.097
Full Name:	Robot Related Work	
Source:	/Users/dodo/Development/ROBOT/paper/robot_related_work.robot	
Start / End / Elapsed:	20240416 20:19:04.310 / 20240416 20:19:28.407 / 00:00:24.097	
Status:	4 tests total, 2 passed, 2 failed, 0 skipped	
TEARDOWN	SeleniumLibrary: Close Browser	00:00:00.168
Documentation:	Closes the current browser.	
Start / End / Elapsed:	20240416 20:19:28.239 / 20240416 20:19:28.407 / 00:00:00.168	
TEST	1 valid test script	00:00:05.444
TEST	2 valid test script	00:00:05.054
Full Name:	Robot Related Work.2 valid test script	
Start / End / Elapsed:	20240416 20:19:10.029 / 20240416 20:19:15.083 / 00:00:05.054	
Status:	FAIL	
Message:	Alert message should have been 'Successfully!' but it was 'Unsuccessfully!'.	
KEYWORD	SeleniumLibrary: Open Browser about:blank, chrome	00:00:02.073
KEYWORD	SeleniumLibrary: Go To \${url}	00:00:00.833
KEYWORD	SeleniumLibrary: Input Text /*[@id="employee_id"], a000000006	00:00:00.314
KEYWORD	SeleniumLibrary: Input Text /*[@id="first_name"], abc2	00:00:00.120
KEYWORD	SeleniumLibrary: Input Text /*[@id="last_name"], def2	00:00:00.142
KEYWORD	SeleniumLibrary: Input Text /*[@id="email"], validemail01@email.com	00:00:00.173
KEYWORD	SeleniumLibrary: Input Text /*[@id="phone_number"], 0900010006	00:00:00.153
KEYWORD	SeleniumLibrary: Input Text /*[@id="age"], 25	00:00:00.141
KEYWORD	SeleniumLibrary: Input Text /*[@id="salary"], 40000	00:00:00.113
KEYWORD	SeleniumLibrary: Select From List By Value /*[@id="department"], Accountant	00:00:00.114
KEYWORD	SeleniumLibrary: Select Radio Button gender, Male	00:00:00.025
KEYWORD	SeleniumLibrary: Select Checkbox /*[@id="overtime"]	00:00:00.105
KEYWORD	SeleniumLibrary: Click Element /*[@id="add"]	00:00:00.075
KEYWORD	SeleniumLibrary: Alert Should Be Present Successfully!	00:00:00.664
Documentation:	Verifies that an alert is present and by default, accepts it.	
Start / End / Elapsed:	20240416 20:19:14.418 / 20240416 20:19:15.082 / 00:00:00.664	
20:19:14.419	INFO	HANDLE: :<class 'NoneType'>: :None
20:19:15.075	INFO	

Figure 15: Test log result from running the related work test script.

```

1 2024-05-19 14:01:15 INFO - Select the JavaScript file. | path="../../../doc/userguide.docx"
2 2024-05-19 14:01:50 Error - Unsupported input file extension. Please select a different file.
3 2024-05-19 14:02:00 INFO - Select the JavaScript file. | path="../../../components/form.js"
4 2024-05-19 14:04:13 INFO - Select the Java Spring Boot project. | path="../../../backend/employee_service"
5 2024-05-19 14:04:54 INFO - Connected to database "employee_service" successfully!
6 2024-05-19 14:05:08 INFO - Select the output path. | path="../../../script/"
7 2024-05-19 14:06:00 INFO - Set filename for output Robot Framework test script. | filename="insert_employee.robot"
8 2024-05-19 14:07:46 INFO - Set filename for output test data. | filename="insert_employee_data.csv"
9 2024-05-19 14:07:49 WARN - Test data file already exists. Please ensure you want to replace this file.
10 2024-05-19 14:08:00 INFO - User pressed OK button.
11 2024-05-19 14:09:00 INFO - User pressed Generate output button.
12 2024-05-19 14:09:05 INFO - Start generating output....
13 2024-05-19 14:12:56 INFO - Robot test script written successfully. | path="../../../script/insert_employee.robot"
14 2024-05-19 14:12:09 INFO - Test Data written successfully. | path="../../../script/insert_employee_data.csv"

```

Figure 16: Log results from running the tool.

Table 7: Details of the number of test cases created for each technique.

Testing technique	Number of valid testcase	Number of invalid testcase	Total Testcase
Boundary Value Testing	28	0	28
Equivalence Class Testing	29	0	29
Robust Boundary Value Testing (proposed approach)	28	14	42
Database Constraints (proposed approach)	5	15	20

This is an example of a stored tool log file. It chronologically records the time, status (success, error, warning), and details of operations. Additionally, for errors and warnings, the log triggers a user-friendly dialog box within the tool window (as shown in Figure 16) to bring these issues to the user's immediate attention.

Test data generation often relies on black-box testing techniques like boundary value testing, equivalence class testing, and robust boundary value testing. However, these techniques may generate data that doesn't align with the system's actual behavior, leading to mismatched expected and actual outputs. This research addresses this challenge by incorporating database constraints into the test data generation process. By ensuring test data adheres to these constraints, we enhance the accuracy of tests and increase the likelihood of uncovering bugs. The effectiveness of this approach can be evaluated by comparing the number of test cases generated using traditional techniques (mentioned above) with the number presented in Table 7.

5 Conclusion

We have outlined an approach for creating frontend and database analysis tool to generate test scripts and test data that adhere to database constraints and robust boundary values. This approach can efficiently identify system errors during software testing by leveraging the generated test data.

In future work, the approach can be enhanced to support additional database constraints such as foreign key constraints, default constraints, and referential integrity constraints. This expansion

would cater to a wider range of software architectures, thereby increasing its utility and applicability.

References

- [1] Ashish Bhargava. 2013. Test Automation in Designing and Implementing Test Automation Frameworks with QTP. Packt Publishing Ltd., 7.
- [2] Ashish Bhargava. 2013. Test automation framework in Designing and Implementing Test Automation Frameworks with QTP. Packt Publishing Ltd., 70.
- [3] Joint Technical Committee ISO/IEC JTC 1, Information technology, Subcommittee SC 7. 2016. Software and systems engineering Software testing, Part 5: Keyword-Driven Testing. 3-18.
- [4] Robot Framework Foundation. 2024. Robot Framework. <https://robotframework.org>
- [5] W3C. 2024. HTML. <https://html.spec.whatwg.org/multipage/introduction.html#a-quick-introduction-to-html>
- [6] W3C. 2024. HTML Elements. <https://html.spec.whatwg.org/multipage/semantics.html#semantics>
- [7] Robot Framework Foundation. 2024. SeleniumLibrary. <https://robotframework.org/SeleniumLibrary/SeleniumLibrary.html>
- [8] Paul Jorgensen. 2014. Software Testing: A Craftsman's Approach, 4 ed. CRC Press. 79-96.
- [9] Ramez Elmasri and Shamkant Navathe. 2016. Database System: The Relational Data Model and Relational Database Constraints, 7 ed. Pearson Ltd. 149-169.
- [10] Pivotal Software. 2024. Spring Framework. <https://spring.io/projects/spring-framework>
- [11] Pivotal Software. 2024. Spring Data REST. <https://spring.io/projects/spring-data-rest>
- [12] Panthakan Sriwichainan and Taratip Suwannasart. 2020. Generating Test Scripts for Web Based Application. SSIP 2020: 2020 3rd International Conference on Sensors, Signal and Image Processing. Prague. Czech Republic. 84-89. <https://dl.acm.org/doi/10.1145/3441233.3441248>
- [13] Kartik Bajaj. 2015. Synthesizing Web Element Locators. 2015 30th IEEE/ACM International Conference on Automated Software Engineering. Lincoln. USA. 331-341. <https://doi.org/10.1109/ASE.2015.23>
- [14] Satish Gojarea, Rahul Joshib and Dhanashree Gaigawarec. 2015. Analysis and Design of Selenium WebDriver Automation Testing Framework. 2nd International Symposium on Big Data and Cloud Computing (ISBCC'15). Chennai. India. 341-346. <https://doi.org/10.1016/j.procs.2015.04.038>